# Project Executable Files

| | |
|---|---|
| **Date** | 15 July 2026 |
| **Team ID** | TM-RISINGWATERS-04 |
| **Project Name** | Rising Waters: AI Powered Flood Prediction System |
| **Maximum Marks** | 3 Marks |

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

## Step 1: Submission Checklist

| S.No | Item to Submit | Submitted (Yes / No / NA) |
|---|---|---|
| 1 | Complete source code (all files and folders) | Yes |
| 2 | README / Setup Guide (instructions to run the project) | Yes |
| 3 | requirements.txt / package.json / dependency file | Yes |
| 4 | Database schema / seed files (if applicable) | Yes |
| 5 | Environment configuration file (.env.example or similar) | Yes |
| 6 | Deployed application URL (if hosted) | Yes |
| 7 | APK / Executable binary (if applicable for mobile/desktop apps) | NA |
| 8 | Dockerfile / Containerization config (if applicable) | Yes |
| 9 | Test files and test results | Yes |
| 10 | Demo video or walkthrough (if required) | Yes |

## Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
/rising-waters-root
├── /src
│   ├── /model                  # PyTorch UNet models and weights
│   ├── /api                    # FastAPI endpoints (predict, telemetry, auth)
│   ├── /static                 # Leaflet.js dashboard web assets
│   ├── /services               # Twilio SMS sender and database connector utilities
│   └── main.py                 # Application entry point
├── /tests                      # PyTest load & integrity mock suites
├── /database
│   └── schema.sql              # PostgreSQL PostGIS structure layout
├── .env.example                # Sample local environments file
├── Dockerfile                  # Multi-stage build setup config
├── README.md                   # Installation & operational details
└── requirements.txt            # Python deep learning and GIS libraries
```

## Step 3: Deployment / Access Details

| Field | Details |
|---|---|
| **Hosted / Deployed URL** | https://rising-waters.apps.dev |
| **Login (Demo) Credentials** | **Role:** Administrator<br>**Username:** admin@risingwaters.org<br>**Password:** DemoFloodPass2026! |
| **Platform / Hosting Provider** | AWS (EC2 instance, RDS PostgreSQL instance, S3 Bucket for GeoTIFF rasters) |
| **Repository Link** | https://github.com/RisingWaters-Team/Flood-Prediction-System |
| **Demo Video Link** | https://youtu.be/dummy-risingwaters-walkthrough |

## Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

```
Pre-requisites:
- Python 3.10+ installed
- PostgreSQL with PostGIS extension enabled
- Twilio developer API tokens (optional for SMS warning delivery)

Step 1: Clone the repository
$ git clone https://github.com/RisingWaters-Team/Flood-Prediction-System.git
$ cd Flood-Prediction-System

Step 2: Install dependencies
$ pip install -r requirements.txt

Step 3: Configure environment variables
$ cp .env.example .env
# Edit .env file to configure database secrets, API keys, and model paths.

Step 4: Run database migrations
$ psql -U postgres -d rising_waters_db -f database/schema.sql

Step 5: Run the server application locally
$ uvicorn src.main:app --host 127.0.0.1 --port 8000 --reload

Step 6: Access the interactive Leaflet Dashboard in your browser
Open http://localhost:8000 in your web browser.
```

## Step 5: Known Issues / Limitations

| S.No | Known Issue / Limitation | Workaround / Status |
|---|---|---|
| 1 | Large Sentinel-2 satellite GeoTIFF files can experience slight memory spikes on models during array operations. | Chunked raster tile processing pipeline was implemented to slice massive files dynamically into 256x256 matrices. |
| 2 | Real-time IoT telemetry synchronization drops slightly if the sensor gateway runs on a weak local connection. | Added dynamic caching buffers inside the backend client to queue sensor logs until a stable signal returns. |
| 3 | The web mapping library Leaflet.js lacks hardware-accelerated 3D terrain shading views directly out of the box. | Simulated elevation shades are rendered server-side and overlaid as flat, styled 2D GeoJSON polygons. |